SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: CSA37 **COURSE NAME: Software Testing**

COURSE OUTCOME COVERED

CO2: Design effective test cases using black-box, white-box testing techniques and to control methods flow. *(BL3)*

Assessment Tool Weightage: 20%

Duration :60 Mins
On Class
QUESTIONS: 5 **Total Marks:50**

Annexure A

Error Analysis Task

Code of Conduct:

I S.Dharmika/192421094 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
S.Dharmika

Annexure A

Error Analysis Task

Q1. Error Analysis of an Online Examination System
Tools: Selenium WebDriver, Chrome DevTools, Excel

A university online examination system has reported errors in student registration, attendance eligibility, and examination access. Some students with less than 75% attendance can access examinations, while eligible students are occasionally rejected.

a) Analyze the reported errors and identify the expected and actual behavior using systematic test design strategies.

b) Apply Equivalence Class Partitioning, Boundary Value Analysis, and Random Testing to identify input-related errors.

c) Design black-box test cases using decision tables and state transition testing to identify errors in examination eligibility and access.

d) Execute the test cases using Selenium WebDriver and document the observed errors with screenshots.

e) Evaluate test adequacy by identifying untested requirements, prioritizing high-risk errors, and recommending additional test cases.

Q2. Error Analysis of a Student Grade Calculator
Tools: Java, JUnit, Maven, JaCoCo

A university grade-calculation application produces incorrect grades for marks near grade boundaries. The source code is available, but the existing test suite does not identify all reported errors.

a) Analyze the source code to identify incorrect conditions, missing validations, and possible logical errors.

b) Apply Equivalence Class Partitioning and Boundary Value Analysis to identify errors in mark validation and grade assignment.

c) Construct a Control Flow Graph (CFG), identify independent paths, and calculate cyclomatic complexity to determine the test paths required for error detection.

d) Develop JUnit test cases and use JaCoCo to analyze statement and branch coverage, identifying uncovered code that may contain defects.

e) Apply mutation testing and evaluate whether the test suite is adequate to detect logical errors and verify the corrected program.

Q3. Error Analysis of an Online Banking Application
Tools: Postman, Swagger/OpenAPI, Excel

An online banking application occasionally processes invalid fund-transfer requests, displays inconsistent error messages, and reports incorrect transaction statuses when network failures occur.

a) Analyze the API requests and responses to identify validation errors, transaction errors, and inconsistencies between expected and actual behavior.

b) Apply black-box test design techniques, including Equivalence Class Partitioning, Boundary Value Analysis, Random Testing, and decision tables, to identify transaction-related errors.

c) Design use-case, state transition, and pairwise test cases to investigate errors involving authentication, transaction status, network type, browser, and repeated transfer requests.

d) Use Postman to execute the test cases and document the errors, including request details, response codes, actual results, and expected results.

e) Evaluate test adequacy using requirement traceability and risk coverage. Explain which internal code-coverage measures cannot be verified when the source code is unavailable.

Q4. Error Analysis of a Commercial Off-The-Shelf (COTS) Learning Management System
Tools: Selenium WebDriver, Apache JMeter, Chrome DevTools

A university purchases a third-party Learning Management System (LMS). Users report assignment-upload failures, browser compatibility problems, and slow responses during peak usage. The source code is not available.

a) Reverse engineer the observable application behavior and identify possible errors in login, course registration, assignment submission, and result viewing.

b) Apply black-box test design techniques, including Equivalence Class Partitioning, Boundary Value Analysis, exploratory testing, and error guessing, to identify functional errors.

c) Use Selenium WebDriver to investigate functional and compatibility errors, and use Apache JMeter to analyze performance-related failures against predefined acceptance criteria.

d) Analyze the limitations of COTS testing and explain why internal Control Flow Graphs, independent execution paths, and source-code coverage cannot normally be measured without suitable source or instrumentation access.

e) Evaluate test adequacy using functional coverage, compatibility results, performance results, and risk analysis. Recommend whether the application satisfies the defined acceptance criteria.

Q5. Error Analysis of a Student Result-Processing Program
Tools: Python, pytest, Coverage.py, Ruff or Pylint

A student result-processing program calculates totals and grades for multiple students. Errors have been reported due to incorrect loop execution, improper variable initialization, invalid mark acceptance, and incorrect pass/fail decisions.

a) Analyze the program logic and identify potential errors involving conditions, loops, variable definitions, and variable uses.

b) Apply boundary-value testing, equivalence-class testing, and error guessing to identify errors involving marks, student counts, subject counts, and invalid inputs.

c) Construct a Control Flow Graph, calculate cyclomatic complexity, and design test cases for independent paths, loop boundaries, and important decision outcomes.

d) Use pytest and Coverage.py to execute the test suite, identify uncovered statements and branches, and investigate whether insufficient coverage is hiding errors. Apply data-flow testing and mutation testing to identify additional logical defects.

e) Evaluate the adequacy of the final test suite using statement coverage, branch coverage, path considerations, data-flow coverage, and mutation results. Recommend further tests before releasing the program.

ANSWERS

Q1. Error Analysis of an Online Examination System

Tools: Selenium WebDriver, Chrome DevTools, Excel

a) Expected and actual behavior

- Expected: Students with attendance of 75% or above and valid registration should access the examination.
- Actual: Students below 75% sometimes gain access, while eligible students are rejected.
- Possible causes: Incorrect attendance conditions, registration validation errors, session errors, or database synchronization issues.

b) ECP, BVA and Random Testing

- Equivalence Class Partitioning (ECP): Divide attendance into valid (75–100%) and invalid (0–74%) classes.
- Boundary Value Analysis (BVA): Test attendance values 74%, 75%, and 76%.
- Random Testing: Generate random attendance values, register numbers, and registration statuses to identify unexpected errors.

c) Decision table and state transition testing

Registration	Attendance	Expected result
Valid	75% or above	Allow access
Valid	Below 75%	Deny access
Invalid	75% or above	Deny access
Invalid	Below 75%	Deny access

State transition: Logged out → Logged in → Eligibility checked → Exam access granted or denied → Exam submitted → Logged out.

d) Selenium execution

1. Open the examination portal using Selenium WebDriver.
2. Enter student login details and attendance test data.
3. Verify the access result against the expected result.
4. Capture screenshots of unexpected access or rejection.
5. Record the test ID, input, expected result, actual result, and status in Excel.

e) Test adequacy

- Check registration, attendance, authentication, and exam submission requirements.
- Prioritize unauthorized exam access as a high-risk defect.
- Add tests for 0%, 74%, 75%, 76%, invalid registration, session expiry, and repeated login attempts.

Q2. Error Analysis of a Student Grade Calculator

Tools: Java, JUnit, Maven, JaCoCo

a) Source-code analysis

Check for incorrect grade conditions, missing mark validation, overlapping grade ranges, and incorrect use of comparison operators. Marks below 0 or above 100 must be rejected.

b) ECP and BVA

Assume the grading scheme is A: 90–100, B: 80–89, C: 70–79, D: 50–69, and Fail: below 50.

Test input	Expected result
-1	Invalid
0	Fail
49	Fail
50	D
69	D
70	C
79	C
80	B
89	B
90	A
100	A
101	Invalid

c) Control Flow Graph and cyclomatic complexity

A typical CFG contains nodes for input validation, grade conditions, grade assignment, and output.

Cyclomatic complexity is:

$V(G)=E-N+2P$

Where E is the number of edges, N is the number of nodes, and P is the number of connected components. For a single connected program, P=1

If there are four independent decision points, V(G)=4+1=5. Therefore, at least five independent paths should be tested. The exact value depends on the actual source code.

d) JUnit and JaCoCo

- Write JUnit tests for valid marks, invalid marks, and grade boundaries.
- Execute tests using Maven.
- Generate a JaCoCo report to measure statement and branch coverage.
- Add tests for uncovered branches.

Example JUnit test:

```
@Test
void testGradeBoundary() {
   assertEquals("A", GradeCalculator.getGrade(90));
   assertEquals("B", GradeCalculator.getGrade(89));
   assertEquals("Invalid", GradeCalculator.getGrade(101));
}
```

Assumption: getGrade() returns a string and uses the grading scheme above.

e) Mutation testing

Introduce small changes, such as replacing >= 90 with > 90, and run the tests again. If the tests fail, the mutation is detected. If they pass, additional tests are needed.

Conclusion: A reliable test suite should cover all grade boundaries, invalid inputs, statements, branches, and important logical mutations.

Q3. Error Analysis of an Online Banking Application

Tools: Postman, Swagger/OpenAPI, Excel

a) Analyze API requests and responses

Check account details, authentication tokens, transfer amounts, account balances, response codes, and transaction status. Compare actual responses with the API specification.

Examples of possible errors:

- Invalid amounts accepted.
- Insufficient-balance transfers processed.

- Incorrect error messages returned.
- Transactions marked successful after network failures.

b) Black-box testing

Technique	Test example	Expected result
ECP	Valid and invalid account numbers	Invalid accounts rejected
BVA	Transfer amount 0, minimum valid amount, and maximum limit	Limits enforced
Random testing	Generate random amounts and account identifiers	Invalid requests rejected
Decision table	Valid login, sufficient balance, valid account	Transfer only when all conditions pass

c) Use-case, state transition and pairwise testing

- Use case: Login → Select beneficiary → Enter amount → Confirm transfer → View receipt.
- State transition: Initiated → Pending → Successful or Failed.
- Pairwise testing: Test combinations of authentication status, browser, network type, and repeated requests.

Verify that retrying a request does not create duplicate transfers.

d) Postman execution

1. Import the API specification into Postman.
2. Send valid and invalid transfer requests.
3. Record request bodies, response codes, response bodies, and transaction IDs.
4. Simulate timeouts or interrupted connections where possible.
5. Save screenshots and results in Excel.

e) Test adequacy

Use requirement traceability to ensure each banking requirement has test cases. Prioritize duplicate transfers, unauthorized access, incorrect balances, and lost transactions.

Since the source code is unavailable, internal statement coverage, branch coverage, and cyclomatic complexity cannot normally be measured. API-level functional and response coverage can still be evaluated.

Q4. Error Analysis of a COTS Learning Management System

Tools: Selenium WebDriver, Apache JMeter, Chrome DevTools

a) Reverse engineering

Observe application behavior for login, course registration, assignment submission, and result viewing. Record input fields, validation messages, navigation, and state changes.

Possible errors include failed uploads, incorrect login messages, missing registrations, and unavailable results.

b) Black-box testing

- ECP: Valid and invalid file formats, login credentials, and course IDs.
- BVA: Minimum and maximum upload size, file-name length, and input limits.
- Exploratory testing: Try different workflows and unexpected user actions.
- Error guessing: Upload duplicate files, disconnect the network, use expired sessions, and submit empty forms.

c) Selenium and JMeter testing

Selenium:

1. Automate login, course registration, assignment upload, and result viewing.
2. Repeat tests in supported browsers.
3. Capture screenshots and browser console errors.

JMeter:

1. Create a test plan with representative user requests.
2. Increase virtual users gradually.
3. Measure response time, throughput, and error percentage.
4. Compare results against predefined acceptance criteria.

d) Limitations of COTS testing

Because source code is unavailable, internal CFGs, independent execution paths, and source-code statement or branch coverage cannot normally be determined. Vendor-provided instrumentation may offer additional visibility.

e) Test adequacy

Evaluate:

- Functional test coverage.
- Browser and device compatibility.
- Response times and failure rates.
- Security and business risks.

Recommendation: Accept the LMS only if it meets the specified functional, compatibility, performance, and security criteria. Otherwise, report defects to the vendor and retest after fixes.

Q5. Error Analysis of a Student Result-Processing Program

Tools: Python, pytest, Coverage.py, Ruff or Pylint

a) Program logic analysis

Check for:

- Incorrect loop limits that skip students or subjects.
- Variables initialized inside the wrong loop.
- Invalid marks accepted.
- Incorrect total and average calculations.
- Incorrect pass/fail conditions.

b) Boundary-value, equivalence-class and error-guessing tests

Assume marks range from 0 to 100 and a pass requires at least 50 marks in each subject.

Input	**Expected result**
-1 mark	Invalid
0 marks	Fail
49 marks	Fail
50 marks	Pass
100 marks	Valid
101 marks	Invalid
0 students	Reject or report no data
Negative subject count	Reject
Missing or non-numeric marks	Validation error

ECP divides inputs into valid and invalid classes. Error guessing checks empty lists, repeated students, missing marks, and incorrect loop initialization.

c) CFG and cyclomatic complexity

Construct a CFG containing input validation, student loop, subject loop, pass/fail decision, and output.

Use:

$V(G)=E-N+2P$

For one connected CFG, $(V(G)=E-N+2)$. If the graph has three independent decision points, its cyclomatic complexity is 4, requiring at least four independent paths. Calculate the actual value from the program's CFG.

Test zero, one, and multiple students, as well as loop entry, loop exit, and pass/fail branches.

d) pytest, Coverage.py and data-flow testing

Example Python function:

def calculate_result(marks): if not marks: raise ValueError("Marks cannot be empty") if any(mark < 0 or mark > 100 for mark in marks): raise ValueError("Invalid marks") total = sum(marks) average = total / len(marks) result = "Pass" if all(mark >= 50 for mark in marks) else "Fail" return total, average, result

Example tests:

import pytestfrom result import calculate_resultdef test_pass(): assert calculate_result([50, 60]) == (110, 55.0, "Pass")def test_fail(): assert calculate_result([49, 80]) == (129, 64.5, "Fail")def test_invalid_marks(): with pytest.raises(ValueError): calculate_result([101])def test_empty_marks(): with pytest.raises(ValueError): calculate_result([])

Run the tests and coverage report:

pytest

coverage run --branch -m pytest

coverage report -m

coverage html

Use Ruff or Pylint to identify suspicious variables, unused values, and code-quality issues. Data-flow testing checks whether variables are correctly defined, used, and updated. Mutation testing checks whether tests detect small changes to conditions or calculations.

e) Test adequacy

- Statement coverage: Measures executed statements.
- Branch coverage: Measures decision outcomes tested.
- Path testing: Checks important execution paths.
- Data-flow coverage: Checks variable definitions and uses.
- Mutation testing: Checks whether tests detect injected defects.

Recommendation: Add tests for uncovered branches, invalid inputs, loop boundaries, incorrect totals, and grade decisions. Release the program only after critical defects are fixed and the complete test suite passes.

Overall conclusion: Systematic testing combines input partitioning, boundary testing, decision tables, state transitions, structural coverage, and risk-based prioritization to identify defects and improve software reliability.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand